

Unit - 1

Introduction - Algorithm Design:

An algorithm is any well defined computational procedure that takes some value or set of values as input and produces some value or set of values as output.

An algorithm is a sequence of computational steps that transform the input into the output. It is a tool for solving a well specified computational problem.

Fundamentals of Algorithms:

Algorithm must satisfy the following criteria.

- * INPUT - Zero or more quantities are externally supplied
- * OUTPUT - Atleast one quantity is produced.
- * DEFINITENESS - Instruction is clear and unambiguous.
- * FINITENESS - Finite number of steps
- * EFFECTIVENESS - Feasible

Correctness of algorithm:

The algorithm includes following important areas.

1. How to derive algorithm?
2. How to validate algorithm?
3. How to analyze algorithm?
4. How to test a program.

* Debugging

Algorithm specifications:

Algorithm can be described in many ways

- 1) Natural language like English
- 2) Graphical representation called flowcharts
- 3) Pseudocode

Time Complexity Analysis

* Step count - Line count / operation count

* Executable statement - Line count $\Rightarrow c1$

Non executable statement - No. of steps $\Rightarrow c0$

Algorithm:

Algorithm simple (A, B, C)

Begin

A = B + 1

C = A + 2

D = A + B

end.

STEP	PROGRAM	STEP / EXECUTION	FREQUENCY	TOTAL
1	Algorithm simple (A, B, C)	0	-	-
2	Begin	0	-	-
3	A = B + 1	1	1	1
4	C = A + 2	1	1	1
5	D = A + B	1	1	1
6	End	0	-	-
Total			-	3

Algorithm sum()

Begin

sum = 0

for i = 1 to n do

sum = sum + i

end for

Return sum

end

$T(n) = 2n + 3 \Rightarrow$ Time complexity.

Insertion sort:

Insertion sort is a simple sorting algorithm that builds the final sorted array one item at a time.

Worst case : $O(n^2)$

Best case : $O(n)$

Average case : $O(n^2)$

Pseudocode: Insertion sort (A)

for i = 1 to n

key $\leftarrow A[i]$

j $\leftarrow i - 1$

while j ≥ 0 and $A[j] > \text{key}$

$A[j+1] \leftarrow A[j]$

j $\leftarrow j - 1$

End while

$A[j+1] \leftarrow \text{key}$

End for.

Line count : Operation count :

To count the number of operations is also known as to analyze the algorithm complexity.

The idea to have a rough idea how many operations are in the worst case needed to executed the algorithm on an input of size n .

Ex : S1 : $prod \leftarrow 0$

S2 : for $i \leftarrow 0$ to $n-1$ do.

S3 : $prod \leftarrow prod + A[i] * B[i]$

S4 : return $prod$.

L1 : Assignment = 1

L2 : Loop = 1

L3 : 5 c/p per iteration = $5n$

(mul, add, 2 array ref, assign)

L4 : Return = 1

ST

Loop : increment $\Rightarrow$ Add & Assign = $2n$

termination $(n+1)$

success / failure

$$\begin{aligned} \text{Total} &\Rightarrow 1 + 1 + 5n + 1 + 2n + n + 1 \\ &= 4 + 8n. \end{aligned}$$

Algorithm Design Paradigms :

Paradigms - pattern of thought.

Various algorithm paradigm

- * Brute force
- * Divide and conquer
- * Backtracking
- * Greedy
- * Dynamic programming

Designing an algorithm:

1. Understanding the problem
2. Decision making on
 - capabilities of computational devices
 - choices of either exact or approximate
 - data structures
 - Algorithmic strategies
3. Specification of algorithm
4. Algorithmic verification
5. Analysis of algorithm
6. Implementation or coding of algorithm.

Analysis of algorithm:

Generally an algorithm is analysed based on its
executing time and amount of space.
space complexity $\Rightarrow$ Amount of memory
Time complexity $\Rightarrow$ Amount of computer time

Introduction - Algorithm Design:

An algorithm is any well defined computational procedure that takes some value or set of values as input and produces some value or set of values as output.

An algorithm is a sequence of computational steps that transform the input into the output. It is a tool for solving a well specified computational problem.

Fundamentals of Algorithms:

Algorithm must satisfy the following criteria.

- * INPUT - Zero or more quantities are externally supplied.
- * OUTPUT - Atleast one quantity is produced.
- * DEFINITENESS - Instruction is clear and unambiguous.
- * FINITENESS - Finite number of steps
- * EFFECTIVENESS - Feasible.

Correctness of algorithm:

The algorithm includes following important areas.

1. How to derive algorithm?
2. How to validate algorithm?
3. How to analyze algorithm?
4. How to test a program.
 - * Debugging
 - * Profiling or performance measurement.

Algorithm specifications:

Algorithm can be described in many ways

- 1) Natural language like English
- 2) Graphic representation called flowcharts
- 3) Pseudocode.

Fin

Time complexity Analysis:

- * Step count - line count / operation count
 - * Executable statement - line count $\Rightarrow 01$
- Non executable statement - No. of steps $\Rightarrow 0$

Algorithm:

Algorithm simple (A, B, C)

Begin

A = B + 1

C = A + 2

D = A + B

end.

STEP	PROGRAM	STEP / EXECUTION	FREQUENCY	TOTAL
1	Algorithm simple (A, B, C)	0	-	-
2	Begin	0	-	-
3	A = B + 1	1	1	1
4	C = A + 2	1	1	1
5	D = A + B	1	1	1
6	End	0	-	-
			Total	3

Algorithm sum()

Begin

sum = 0.0

— 1

for i = 1 to n do

— n+1

sum = sum + 1

— n

end for

— 0

Return sum

— 1

end

— 0

$T(n) = 2n + 3 \Rightarrow$ Time complexity.

Insertion sort:

Insertion sort is a simple sorting algorithm that builds the final sorted array one item at a time.

Worst case : $O(n^2)$

Best case : $O(n)$

Average case : $O(n^2)$

Pseudocode: Insertion sort (A)

for i = 1 to n

key $\leftarrow A[i]$

j $\leftarrow i + 1$

while j > 0 and $A[j] > \text{key}$

$A[j+1] \leftarrow A[j]$

j $\leftarrow j - 1$

End while

$A[j+1] \leftarrow \text{key}$

End for.

Line count ; Operation count :

To count the number of operations is also known as to analyze the algorithm complexity.

The idea to have a rough idea how many operations are in the worst case needed to executed the algorithm on an input of size n .

Ex : S1 : prod $\leftarrow$ 0

S2 : for $i \leftarrow 0$ to $n-1$ do

S3 : prod $\leftarrow$ prod + A[i] * B[i]

S4 : return prod.

L1 : Assignment = 1

L2 : Loop = 1

L3 : 5 op per iteration = $5n$

(mul, add, 2 array ref, assign)

L4 : Return = 1

Loop : increment $\Rightarrow$ Add * Assign = $2n$

termination ($n+1$)

success / failure

$$\begin{aligned} \text{Total} &\Rightarrow 1 + 1 + 5n + 1 + 2n + n + 1 \\ &= 4 + 8n. \end{aligned}$$

Algorithm Design Paradigms :

Paradigms - Pattern of thought.

Various algorithm paradigm

- * Brute force
- * Divide and conquer
- * Backtracking
- * Greedy
- * Dynamic programming.

Designing an algorithm:

1. Understanding the problem
2. Decision making on
 - capabilities of computational devices
 - choices of either exact or approximate
 - data structures
 - Algorithmic strategies.
3. Specification of algorithm
4. Algorithmic verification
5. Analysis of algorithm
6. Implementation or coding of algorithm.

Analysis of algorithm:

Generally an algorithm is analysed based on its
executing time and amount of space.
space complexity $\Rightarrow$ Amount of memory
Time complexity $\Rightarrow$ Amount of computer time.

Efficiencies of algorithm :

3 cases - (1) Best case

(2) Worst case

(3) Average case.

Worst - case :

* The longest running time for any input of size n .

* Upper bound on the running time.

* Ex : Sort a set of numbers in increasing order.

Best case :

Sort a set of numbers in increasing order and the data is already in increasing order.

Average case :

Number of times the basic operation will be executed on typical input.

Worst case : $W(n)$ - Maximum over inputs of size n .

Best case : $B(n)$ - Minimum over inputs of size n .

Average case : $A(n)$ - Average over inputs of size n .

Asymptotic notations based on growth function :

The growth of the running time of an algorithm gives a simple characterization of the algorithm's efficiency. Asymptotic notation is shorthand way to represent the time complexity. Various notations are Ω , Θ and O (Best, Average, worst).

Big-Oh Notation (O):

- * Method of representing the upper bound of algorithm's running time.
- * Gives largest amount of time taken by the algorithm to complete.

Definition:

Let $F(n)$ and $g(n)$ be two non-negative functions.

Let n_0 and constant c are two integers such that n_0 denotes some value of input $n > n_0$

$$F(n) \leq c * g(n)$$

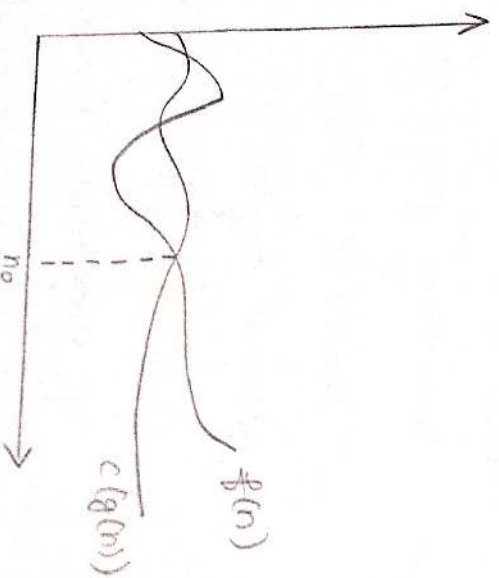
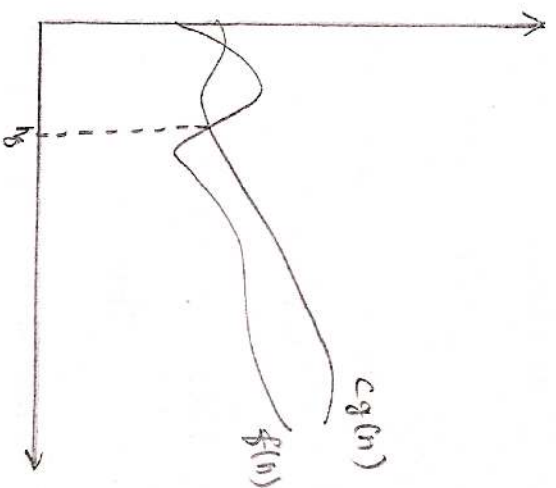
Big-omega Notation: (Ω)

used to represent the lower bound of algorithm's running time. We denote shortest amount of time taken by algorithm.

Definition:

Let $F(n)$ is said to be in $\Omega(g(n))$ if $F(n)$ is bounded below by some positive constant multiple of $g(n)$.

$$F(n) \geq c * g(n)$$



Big Theta Notation (Θ)

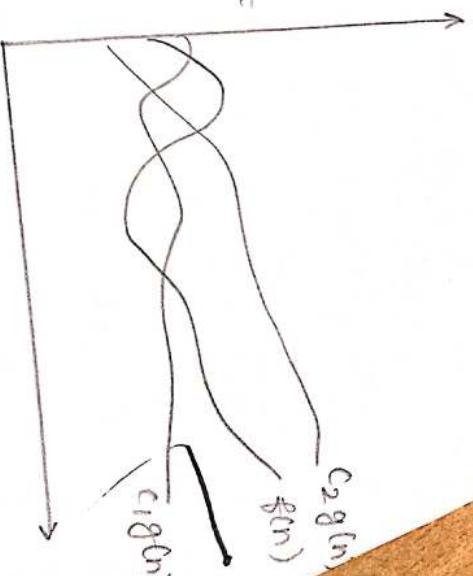
Running time between upper bound and lower bound.

Let $f(n)$ and $g(n)$ be two positive functions. There are two positive constants namely c_1 and c_2 such that

$$c_1 g(n) \leq f(n) \leq c_2 g(n)$$

Thus we can say that

$$f(n) \in \Theta(g(n))$$



Little Oh Notation:

This is denoted as O . This represents a loose bounding version of Big O . $g(n)$ bounds from the top, but it does not bound the bottom.

Little omega Notation:

It bounds from the bottom, but not the top.

$$\lim_{n \rightarrow \infty} \frac{g(n)}{f(n)} = 0$$

Mathematical Analysis:

Mathematical Induction:

- Inductive proof are the special kind of proof called 'Inductive' that is essential when dealing with recursively defined objects
- Mathematical induction can be used to prove a sequence of statements indexed by positive integers.

Q: Prove that $\sum_{i=1}^n i = \frac{n(n+1)}{2}$ for all $n \geq 1$

Basis: $n=1$ $\sum_{i=1}^1 i = \frac{1(1+1)}{2}$

$$= \frac{2}{2} = 1$$

The truth is immediate since both sides are equal.

Inductive step:

Assume that the equation is true for n and prove that

it is true for $n+1$.

Assume $\sum_{i=1}^n i = \frac{n(n+1)}{2}$, P.T $\sum_{i=1}^{n+1} i = \frac{(n+1)(n+2)}{2}$

$$\sum_{i=1}^{n+1} i = \left(\sum_{i=1}^n i \right) + (n+1)$$

$$= \frac{n(n+1)}{2} + (n+1)$$

$$\frac{(n+1)(n+2)}{2} = \frac{n(n+1) + 2(n+1)}{2}$$

$$\frac{n^2 + 2n + 2}{2} = \frac{n^2 + n + 2n + 2}{2}$$

$$\frac{n^2 + 3n + 2}{2} = \frac{n^2 + 3n + 2}{2}$$

$$\text{LHS} = \text{RHS.}$$

hence proved ie, $\sum_{i=1}^n i = \frac{n(n+1)}{2} \quad \forall n \geq 1.$

Recurrence relations:

A recurrence is an equation or inequality that describes a function in terms of its value on smaller inputs.

3 Methods to solve recurrence.

- 1) substitution,
- 2) Recursion tree method
- 3) Master Method.

Substitution Method:

The substitution method for solving recurrence has 2 steps.

1. Guess the form of the solution.
2. Use mathematical induction to find the constants and show that the solution works.

2 Types:

1. Forward substitution
2. Backward substitution.

Forward substitution:

Ex: Consider a recurrence relation $T(n) = T(n-1) + n$ with initial condition $T(0) = 0$

Let $T(n) = T(n-1) + n$

If $n=1$, then $T(1) = T(0) + 1$

$$\boxed{T(1) = 1} \quad \text{--- } 0+1$$

If $n=2$ then $T(2) = T(1) + 2$
 $= 1 + 2$

$$\boxed{T(2) = 3}$$

If $n=3$ then $T(3) = T(2) + 3$
 $= 3 + 3$

$$\boxed{T(3) = 6}$$

By observing above generated equations we can devise a formula,

$$T(n) = \frac{n(n+1)}{2} = \frac{n^2}{2} + \frac{n}{2}$$

Denote $T(n)$ in terms of bigth notation.

$$\boxed{T(n) = O(n^2)}$$

Backward substitution Method:

Ex: Consider a recurrence relation,

$T(n) = T(n-1) + 1 \rightarrow \textcircled{1}$ with initial condition $T(0) = 0$

$T(n-1) = T(n-1-1) + (n-1) \rightarrow \textcircled{2}$

Putting eqn. $\textcircled{2}$ in eqn. $\textcircled{1}$ we get,

$$T(n) = T(n-2) + (n-1) + n \rightarrow \textcircled{3}$$

Let $T(n-2) = T(n-2-1) + (n-2) \rightarrow \textcircled{4}$

Put, eqn. $\textcircled{4}$ in eqn. $\textcircled{3}$ We get

$$T(n) = T(n-3) + (n-2) + (n-1) + n.$$

$$= T(n-k) + (n-k+1) + (n-k+2) + \dots + n.$$

if $k=n$, then

$$T(n) = T(0) + 1 + 2 + \dots + n$$

$$T(n) = 0 + 1 + 2 + \dots + n$$

$$T(n) = \frac{n(n+1)}{2}$$

$$T(n) = \frac{n^2}{2} + \frac{n}{2}$$

Denote $T(n)$ in terms of big oh,

$$T(n) \in O(n^2)$$

Recurrence Tree:

- * Each node represent the cost of a single subproblem.
- * Sum the cost within each level of the tree to obtain a set of per level costs and then sum all the prelevel costs to determine the total cost of all level of the recursion.

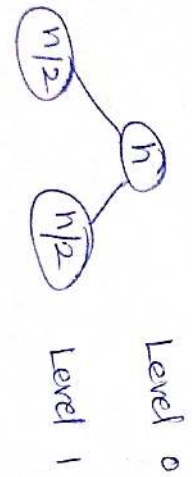
* Recursion trees are particularly useful when the recurrence describes the running time of a divide and conquer algorithm.

Example: $T(n) = 2T(n/2) + n$

one problem;

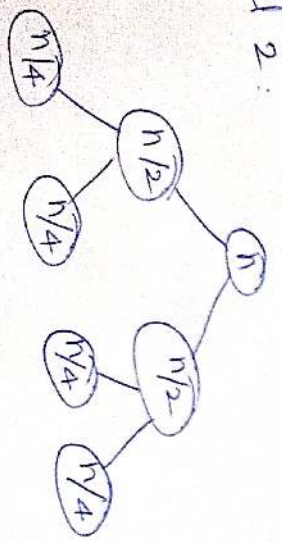


Recurrence tree at level 1



cost
 $\frac{n}{2} + \frac{n}{2} = n$

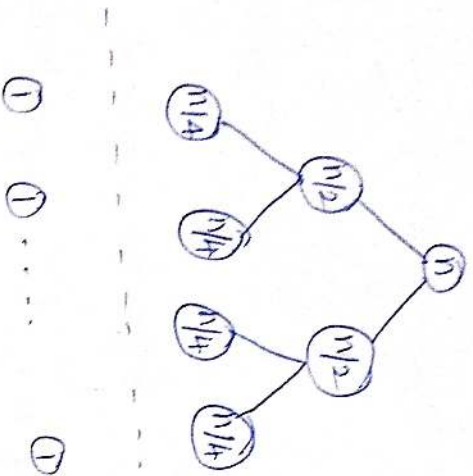
Level 2:



cost
 $\frac{n}{2} + \frac{n}{2} = n$

cost
 $\frac{n}{4} + \frac{n}{4} + \frac{n}{4} + \frac{n}{4} = n$

Final Recurrence Tree :



Levels	No. of prob.	Problem size	Workdone
0	1	n	$1 \times n = n$
1	2	$n/2$	$2 \times \frac{n}{2} = n$
2	4	$n/4$	$4 \times \frac{n}{4} = n$
3			
⋮			
$\log(n)$	$2^{\log(n)}$	$\frac{n}{2^{\log(n)}}$	$2^{\log(n)} \times \frac{n}{2^{\log(n)}} = n$

Total $(1 + \log(n)) \times n$

Time complexity

$$2^{\log n} \times T(1) = n^{\log_2 2} = n \times 1 = n$$

$$(1 + \log_2(n)) \times n = n \log_2 n + n$$

$$\approx O(n \log n)$$

_____ x _____

$$T(n) = \begin{cases} b & n \leq 2 \\ 8T(n/2) + cn^2 & n > 2 \end{cases} \quad b, c \Rightarrow \text{constants}$$

(ii).

$$\therefore T(n) = O(n^3).$$

The recurrence eqn. $T(n)$ is derived from Völler's theorem by

$$T(n) = \begin{cases} b & n \leq 2 \\ 7T(n/2) + an^2 & n \geq 2 \end{cases} \quad a, b \rightarrow \text{constants}$$

$$\begin{aligned} \therefore T(n) &= an^2 + \left[1 + 7/4 + (7/4)^2 + \dots + (7/4)^{k-1}\right] + 7^k T(1) \\ &\leq Cn^2 (7/4) \log_2 n + 7 \log_2 n \\ &= O(n \log_2 7) \\ &\approx O(n^{2.81}) \end{aligned}$$

Master theorem Proof:

Let the time complexity fn. $T(n)$ be a positive & eventually non-decreasing fn. of the following form,

$$T(n) = aT(n/b) + Cn^k$$

$$T(1) = d$$

Here $a, d, b \in k \rightarrow \text{const.}$ Here $b \geq 2, k \geq 0, a > 0, c > 0 \& d \geq 0$

case (i) $T(n) = O(n^k)$ if $a < b^k$

(ii) $T(n) = \Theta(n^k \log n)$ if $a = b^k$

(iii) $T(n) = \Theta(n \log_b a)$ if $a > b^k$

Proof: To prove Master's Theorem:

$$T(n) = aT(n/b) + Cn^k$$

$$\text{Total cost } T(n) = Cn^k \left[1 + \frac{a}{b^k} + \dots + \left(\frac{a}{b^k}\right)^2 + \dots + \left(\frac{a}{b^k}\right)^{\log_b n}\right]$$

$$\text{Rewritten as, } T(n) = Cn^k [1 + d + d^2 + \dots + d \log_b n]$$

$$\text{Here } d = (a/b^k)$$

case (i) when $a < b^k$ i.e., $d < 1$

$$1 + d + d^2 + \dots = \frac{1}{1-d}$$

$$\therefore T(n) = C_n^k \left[\frac{1}{1-d} \right]$$

$$= \frac{C_n^k}{1-d}$$

$$T(n) = O(n^k)$$

case (ii) when $a = b^k$ (i.e.) $d = 1$

$$T(n) = C_n^k [1 + 1 + \dots + 1 \log_b n]$$

$$= C_n^k [1 + \log_b n]$$

$$T(n) = O(n^k \log_b n)$$

case (iii) $a > b^k$ i.e. $d \neq 1$

$$T(n) = C_n^k d \log_b n \left[\left(\frac{1}{d} \right) \log_b n + \dots + \frac{1}{d} + 1 \right]$$

$$= C_n^k \times \left(\frac{a}{b^k} \right) \log_b n$$

$$= C \times a \log_b n$$

$$T(n) = O(n \log_b a)$$

Master Theorem Example (simplified)

$$1. T(n) = 8T(n/2) + n^2$$

This is of the form: $T(n) = aT(n/b) + f(n)$

$$a = 8, b = 2, f(n) = n^2$$

$$n \log a = n \log 8$$

$$= n \log_2 8$$

$$= n^3 \log_2 2$$

$$= n^3$$

From theorem,

$$f(n) = O(n \log_b a - \epsilon)$$

$$= O(n^3 |n|) = O(n^2)$$

$$\text{case (i)} T(n) = O(n \log_b a)$$

$$T(n) = O(n^3)$$

Generalized Master Theorem:

$$T(n) = aT(n/b) + n^2$$

Recurrence of the form

$$T(n) = aT(n/b) + f(n)$$

Here $a=8, b=2, f(n)=n^2$

Compare $f(n)$ with factors $n \log a$

$$\begin{aligned} n \log a &= n \log 8 = n \log_2 8 \\ &= n^3 \log_2 2 \\ &= n^3 \end{aligned}$$

$$= O(n^3 \log n)$$

$$f(n) = O(n^2)$$

$$\begin{aligned} \text{Case (i)} \quad T(n) &= O(n \log b^a) \\ &= O(n \log_2 8) \end{aligned}$$

$$T(n) = O(n^3)$$

Find the Maximum & Minimum in array:

Algorithm MaxMin(i, j, max, min)

// $a[i:n]$ is a global array. Parameters i & j are integers

// $1 \leq i \leq j \leq n$. The effect is to set max & min to the largest

// & smallest values in $a[i:j]$ resp.

{ if $(i=j)$ then $\text{max} = \text{min} = a[i]$; // small(p)

else if $(i=j-1)$ then // Another case of small(p)

{ if $(a[i] < a[j])$ then

{ $\text{max} = a[j]$;

$\text{min} = a[i]$;

}

else

{ $\text{max} = a[i]$; $\text{min} = a[j]$;

}

else

{ $\text{mid} = \lfloor (i+j)/2 \rfloor$;

MaxMin(i, mid, max, min);

MaxMin(mid+1, j, max, min);

if $(\text{max} < \text{max1})$ then $\text{max} = \text{max1}$;

if $(\text{min} > \text{min1})$ then $\text{min} = \text{min1}$;

Examples.

Time Complexity Analysis:

$$T(n) = \begin{cases} T(\lfloor n/2 \rfloor) + T(\lfloor n/2 \rfloor) + 2, & n > 2 \\ n = 2 \\ n = 1 \end{cases}$$

$n = 2^k$ for some +ve int k . Then,

$$\begin{aligned} T(n) &= 2T(n/2) + 2 \\ &= 2(2T(n/4) + 2) + 2 \\ &= 4T(n/4) + 4 + 2 \\ &= 2^{k-1}T(2) + \sum_{i=1}^{k-1} 2^i \\ &= 2^{k-1} + 2^k - 2 \end{aligned}$$

$T(n) = \frac{3n}{2} - 2$ is the best, average & worst case.

Algorithm for finding closest pair:

The problem is to find out the closest pair of points in the array, among n points of a set S .

$P(x_1, y_1)$ & $Q(x_2, y_2)$

$$d(P, Q) = \sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2}$$

The set S is divided into two subsets, S_{left} & S_{right} using a

vertical line L .

$\lfloor |S|/2 \rfloor$ and $\lceil |S|/2 \rceil \rightarrow$ Two subsets S_{left} & S_{right} .

Algorithm ClosestPair $S[1 \dots n]$

// Input: A set of n points, S

// Output: Two closest points & distance

Begin

If $n < \text{threshold}$, then solve the problem by conventional algorithm.

else $i = 1, j = n$

$\text{mid} = \lfloor \frac{i+j}{2} \rfloor$

$d_1 = \text{closest pair}(S[1 \dots \text{mid}])$

$d_2 = \text{closest pair}(S[\text{mid}+1 \dots n])$

$d = \min(d_1, d_2)$
end if